

CS432: Databases

Relational Database Design

Instructor
Yogesh K. Meena

Lecture no.
15

CSE, IIT Gandhinagar
March 18, 2026

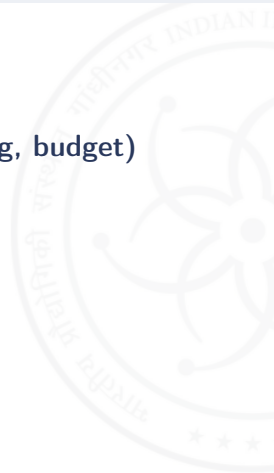
University Schema

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

Design Alternative

Larger Schemas

No instructor and department schemas. We have only:
inst_dept (ID, name, salary, dept_name, building, budget)



Design Alternative

Larger Schemas

No instructor and department schemas. We have only:
inst_dept (ID, name, salary, dept_name, building, budget)

Redundancy

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Design Alternative

Larger Schemas

No instructor and department schemas. We have only:

inst_dept (ID, name, salary, dept_name, building, budget)

Incompleteness

- Create a new department in the university with no instructor.
- May need to add NULL values

Design Alternative

Smaller Schemas

Start with with the schema `inst_dept`.

Discovering repetition

- A real-world database has a large number of schemas and an even larger number of attributes.
- The number of tuples can be in the millions or higher.
- Discovering repetition is **costly**.
- Sometimes, lack of repetition is just a “lucky” special case.

May need to approach enterprise itself to understand its rules

Design Alternative

Decompositions of schemas

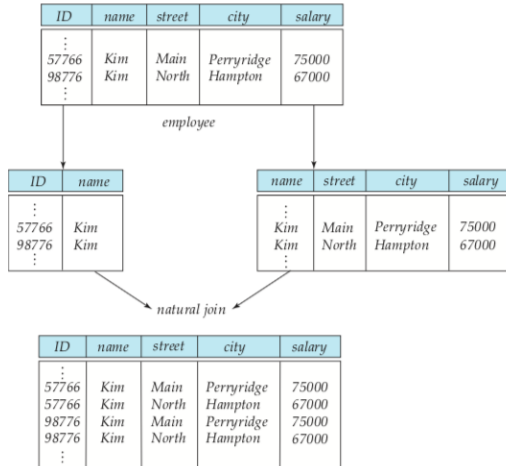
employee (ID, name, street, city, salary)

employee1 (ID, name)

employee2 (name, street, city, salary)

Design Alternative

Lossy decompositions



Atomic Domains and First Normal Form

- A domain is **atomic** if elements of the domain are considered to be indivisible units.
- Relation schema **R** is in **first normal form (1NF)** if the domains of all attributes of R are atomic.
- Add an attribute **children** in **employee** schema. **Not in 1NF**
- Integers are assumed to be atomic. Set of all sets of integers is a nonatomic domain.

Atomic Domains and First Normal Form

Consider an alternative way to write Employee ID.

- The first two letters: department
- Remaining four digits: Unique number within the department
- For example: “CS001” and “EE1127”.

Atomic Domains and First Normal Form

Consider an alternative way to write Employee ID.

- The first two letters: department
- Remaining four digits: Unique number within the department
- For example: “CS001” and “EE1127”.
- **Nonatomic**, can be divided into smaller units.

Potential Problems?

Atomicity in real scenerios

- Composite-valued, multivalues attributes are often useful in many cases.
- Atomicity is not always desirable.
- Huge runtime overhead of converting data back and forth from the atomic form.
- Modern database systems do support many types of nonatomic values.
- We restrict ourselves to relations in first normal form and, thus, all domains are atomic.

Functional Dependencies

Let $r(R)$ be the schema for relation r , with R denoting the set of attributes and $\alpha \subseteq R$ and $\beta \subseteq R$

Given an instance of $r(R)$, we say that the instance satisfies the functional dependency $\alpha \rightarrow \beta$ if for all pairs of tuples t_1 and t_2 in the instance such that $t_1[\alpha] = t_2[\alpha]$, it is also the case that $t_1[\beta] = t_2[\beta]$

An example instance

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4

- $A \rightarrow C$
- $C \rightarrow A$
- $A \rightarrow D$

An example instance

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4

- $A \rightarrow C$ Satisfied
- $C \rightarrow A$ Not satisfied
- $A \rightarrow D$ Not satisfied

Keys and Functional Dependencies

K is a superkey of $r(R)$ if the functional dependency $K \rightarrow R$ holds on $r(R)$.

K is a superkey if, for every legal instance of $r(R)$, for every pair of tuples t_1 and t_2 from the instance, whenever $t_1[K] = t_2[K]$, it is also the case that $t_1[R] = t_2[R]$.

Functional Dependencies

Trivial Functional Dependencies

- $A \rightarrow A$
- $AB \rightarrow A$

In general, $\beta \subseteq \alpha$.

Closure of Functional Dependencies

F^+ to denote the closure of the set F . It is the set of all functional dependencies that can be inferred given the set F .

Boyce-Codd Normal Form

A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency
- α is a superkey for schema R .

Boyce-Codd Normal Form

A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency
- α is a superkey for schema R .

Example:

inst_dept (ID, name, salary, dept_name, building, budget)

In BCNF?

Decomposing schema that are not in BCNF

If a relation R is not in BCNF, then there is at least one nontrivial functional dependency $\alpha \rightarrow \beta$ such that α is not a superkey for R . Replace R in our design with two schemas:

- $\alpha \cup \beta$
- $(R - (\beta - \alpha))$

Decomposing schema that are not in BCNF

If a relation R is not in BCNF, then there is at least one nontrivial functional dependency $\alpha \rightarrow \beta$ such that α is not a superkey for R . Replace R in our design with two schemas:

- $\alpha \cup \beta$
- $(R - (\beta - \alpha))$

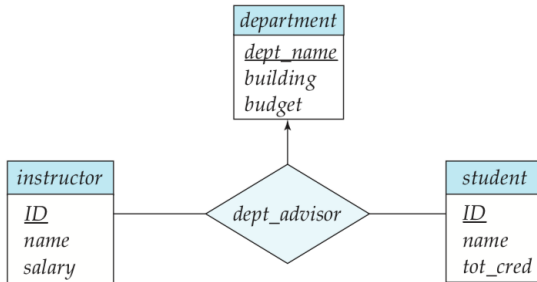
Example

inst_dep $\alpha = \text{dept_name}$, $\beta = \text{building, budget}$

- $\alpha \cup \beta = (\text{dept_name, building, budget})$
- $(R - (\beta - \alpha)) = (\text{ID, name, dept_name, salary})$

BCNF and Dependency Preservation

A student may have more than one advisor, but at most one corresponding to a given department.



dept_advisor(s_ID, i_ID, dept_name)

BCNF and Dependency Preservation

An additional constraint: An instructor can act as advisor for only a single department.

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

Is dept_advisor in BCNF?

BCNF and Dependency Preservation

An additional constraint: An instructor can act as advisor for only a single department.

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

Is dept_advisor in BCNF?

Decomposition

(s_ID, i_ID)

$(i_ID, dept_name)$

BCNF and Dependency Preservation

An additional constraint: An instructor can act as advisor for only a single department.

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

Is dept_advisor in BCNF?

Decomposition

(s_ID, i_ID)

$(i_ID, dept_name)$ No schema that includes all attributes in the second functional dependency.

Third Normal Form

- BCNF requires that all nontrivial dependencies be of the form $\alpha \rightarrow \beta$, where α is a superkey.
- Third normal form (3NF) relaxes this constraint.

It says atleast one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency
- α is a superkey for schema R.
- Each attribute A in $(\beta - \alpha)$ is contained in a candidate key for R.

Any schema that satisfies BCNF also satisfies 3NF

Revisiting dept_advisor relationship

$i_ID \rightarrow dept_name$ caused dept_advisor not in BCNF.

Is dept_advisor 3NF?

Higher Normal Forms

(ID, child name)

(ID, phone_number)

Combine these schemas: (ID, child name, phone_number)

Is above schema in BCNF?

Higher Normal Forms

(ID, child name)

(ID, phone_number)

Combine these schemas: (ID, child name, phone_number)

Is above schema in BCNF?

Example

ID: 99999

Two children: “David” and “William”

Two phone numbers: 512-555-1234 and 512-555-4321.

Higher Normal Forms

(ID, child name)

(ID, phone_number)

Combine these schemas: (ID, child name, phone_number)

Is above schema in BCNF?

Example

ID: 99999

Two children: "David" and "William"

Two phone numbers: 512-555-1234 and 512-555-4321.

(99999, David, 512-555-1234)

(99999, David, 512-555-4321)

(99999, William, 512-555-1234)

(99999, William, 512-555-4321)

Closure of a Set of Functional Dependencies

Relation schema

- $r(A, B, C, G, H, I)$

Functional dependencies set

$A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H$

$A \rightarrow H?$

Logically Implication

Given a relational schema $r(R)$, a functional dependency f on R is **logically implied** by a set of functional dependencies F on r if every instance of $r(R)$ that satisfies F also satisfies f .

Logically Implication

Given a relational schema $r(R)$, a functional dependency f on R is **logically implied** by a set of functional dependencies F on r if every instance of $r(R)$ that satisfies F also satisfies f .

$A \rightarrow H$ is logically implied.

Closure

The closure of F , denoted by F^+ , is the set of all functional dependencies logically implied by F .

Finding logically implied functional dependencies

Armstrong's axioms

- **Reflexivity rule.** If α is a set of attributes and $\beta \subseteq \alpha$ then $\alpha \rightarrow \beta$ holds.
- **Augmentation rule.** If $\alpha \rightarrow \beta$ holds and γ is a set of attributes, then $\gamma\alpha \rightarrow \gamma\beta$ holds.
- **Transitivity rule.** If $\alpha \rightarrow \beta$ holds and $\beta \rightarrow \gamma$ holds, then $\alpha \rightarrow \gamma$ holds.

They are sound and complete.

Finding logically implied functional dependencies

List of additional rules

- **Union rule.** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule.** If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule.** If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Finding logically implied functional dependencies

List of additional rules

- **Union rule.** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule.** If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule.** If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Example

$R = (A, B, C, G, H, I), F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

$A \rightarrow H$

$CG \rightarrow HI$

$AG \rightarrow I$

Finding logically implied functional dependencies

List of additional rules

- **Union rule.** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule.** If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule.** If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Example

$R = (A, B, C, G, H, I)$, $F = A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H$

$A \rightarrow H$ Transitivity rule

$CG \rightarrow HI$ Union rule

$AG \rightarrow I$ Pseudotransitivity rule

A procedure to compute F^+

$F^+ = F$

repeat

for each functional dependency f in F^+

 apply reflexivity and augmentation rules on f

 add the resulting functional dependencies to F^+

for each pair of functional dependencies f_1 and f_2 in F^+

if f_1 and f_2 can be combined using transitivity

 add the resulting functional dependency to F^+

until F^+ does not change any further

Closure of Attribute Sets

Attribute B is functionally determined by α if $\alpha \rightarrow B$.

Set of all attributes functionally determined by α under a set F of functional dependencies the closure of α under F (α^+).

```
result :=  $\alpha$ ;  
repeat  
    for each functional dependency  $\beta \rightarrow \gamma$  in  $F$  do  
        begin  
            if  $\beta \subseteq \textit{result}$  then  $\textit{result} := \textit{result} \cup \gamma$ ;  
        end  
until ( $\textit{result}$  does not change)
```

compute $(AG)^+$

Canonical Cover

To find a simplified set of functional dependencies that has the same closure as the given set.

Extraneous attributes

The attribute that can be removed without changing the closure of the set of functional dependencies.

- Attribute A is extraneous in α if $A \in \alpha$, and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$, and $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .

Example 1: $AB \rightarrow C$ and $A \rightarrow C$

Example 2: $AB \rightarrow CD$ and $A \rightarrow C$

Canonical Cover

$F_c = F$

repeat

Use the union rule to replace any dependencies in F_c of the form $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$.

Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous attribute either in α or in β .

/* Note: the test for extraneous attributes is done using F_c , not F */

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$ in F_c .

until (F_c does not change)

A canonical cover F_c for F is a set of dependencies such that F logically implies all dependencies in F_c , and F_c logically implies all dependencies in F .

Exercise

Consider the following set F of functional dependencies on schema (A, B, C) :

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$A \rightarrow B$$

$$AB \rightarrow C$$

Find the canonical cover for F .

Exercise

Consider the following set F of functional dependencies on schema (A, B, C) :

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$A \rightarrow B$$

$$AB \rightarrow C$$

Find the canonical cover for F .

1. We combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$.

Exercise

Consider the following set F of functional dependencies on schema (A, B, C) :

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$A \rightarrow B$$

$$AB \rightarrow C$$

Find the canonical cover for F .

1. We combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$.
2. A is extraneous in $AB \rightarrow C$ as $B \rightarrow C$ is already present in our set of functional dependencies.

Exercise

Consider the following set F of functional dependencies on schema (A, B, C) :

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$A \rightarrow B$$

$$AB \rightarrow C$$

Find the canonical cover for F .

1. We combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$.
2. A is extraneous in $AB \rightarrow C$ as $B \rightarrow C$ is already present in our set of functional dependencies.
3. C is extraneous in $A \rightarrow BC$, since $A \rightarrow BC$ is logically implied by $A \rightarrow B$ and $B \rightarrow C$.

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar